

Pontifícia Universidade Católica de São Paulo
PUC-SP



PUC-SP

Vinicius de Lucena

Detector de Heliportos em Imagens de Satélite: Treinamento e Avaliação de um
Modelo YOLO26

Ciência de Dados e Inteligência Artificial

São Paulo
2026

SUMÁRIO

1	INTRODUÇÃO	1
2	FUNDAMENTAÇÃO TEÓRICA	1
2.1	DETECÇÃO DE OBJETOS E A FAMÍLIA YOLO	1
3	METODOLOGIA	2
3.1	COLETA DE DADOS	2
4	RESULTADOS E DISCUSSÃO	5
4.1	COMPARAÇÃO DAS RODADAS E DAS ARQUITETURAS	5
4.2	CURVAS DE TREINO	6
5	APLICAÇÃO NO STREAMLIT	8
6	CONCLUSÕES	9
	REFERÊNCIAS	11

1 INTRODUÇÃO

O trabalho desenvolvido aborda o problema da detecção de um objeto específico em imagens de satélite da cidade de São Paulo: o heliponto em cobertura. A visão computacional baseada em Deep Learning permite localizar e classificar objetos em imagens, e neste projeto construí, de ponta a ponta, um detector de uma única classe, cobrindo todo o ciclo de um sistema real: da coleta do dado bruto ao modelo treinado, avaliado e disponibilizado em uma aplicação web.

O objetivo é treinar e avaliar um detector capaz de identificar helipontos e medir sua capacidade de generalizar para uma região não vista durante o treino. Os objetivos específicos são: coletar imagens de forma programática, anotar com consistência, treinar com transfer learning, avaliar com métricas padronizadas, analisar os erros do modelo e disponibilizar uma aplicação de demonstração.

A mensagem central da disciplina orienta o projeto: a maior parte do esforço de um sistema de IA está nos dados, e não na arquitetura. Como o modelo de detecção já vem pré-treinado, o que diferencia um resultado bom de um ruim é a qualidade da coleta, da curadoria e da anotação. Por isso, a maior parte deste relatório trata da construção do conjunto de dados.

2 FUNDAMENTAÇÃO TEÓRICA

Esta seção apresenta os conceitos que fundamentam as decisões técnicas do trabalho.

2.1 DETECÇÃO DE OBJETOS E A FAMÍLIA YOLO

Diferentemente da classificação, em que a rede atribui um rótulo único à imagem inteira, a detecção de objetos precisa responder duas perguntas ao mesmo tempo: o que existe na imagem e onde está cada ocorrência, delimitada por uma caixa (bounding box). O YOLO (You Only Look Once) é uma família de detectores de estágio único que prevê caixas e classes em uma só passagem pela rede, o que o torna rápido e adequado a um fluxo prático de treino e inferência.

O modelo principal escolhido foi o YOLO26n, da Ultralytics, por trazer melhorias úteis para alvos pequenos, como um esquema de atribuição de rótulos consciente de pequenos objetos e uma cabeça de detecção mais leve. A variante nano cabe com folga na GPU T4 do Google Colab e treina rápido para um dataset pequeno. Para a

comparação obrigatória de arquitetura, utilizei também o YOLO11n, sob os mesmos hiperparâmetros.

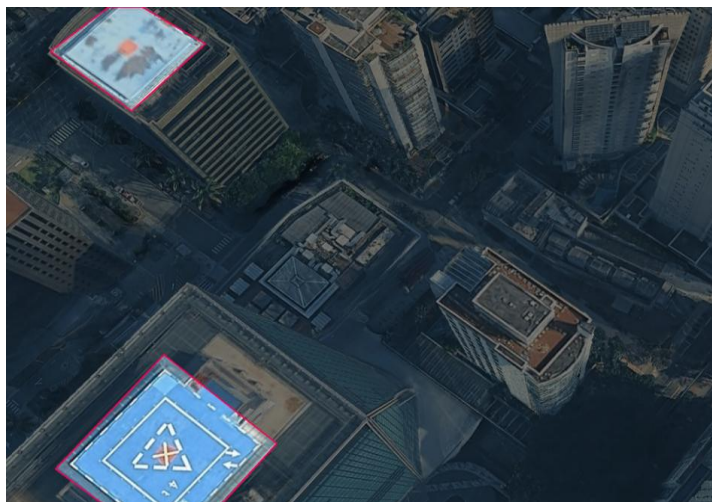


Figura 1 - Exemplo de heliponto em vista de satélite, com a caixa de anotação

2.2 MÉTRICAS DE DETECÇÃO (IOU, NMS, PRECISÃO, REVOCACÃO E mAP)

A avaliação de um detector se apoia em algumas definições vistas em aula:

- **IoU (Intersection over Union):** mede a sobreposição entre a caixa prevista e a caixa verdadeira; é o critério que decide se uma detecção "casa" com um objeto real.
- **NMS (Non-Maximum Suppression):** elimina detecções redundantes sobre o mesmo objeto, mantendo a de maior confiança.
- **Precisão:** das detecções feitas, quantas estavam corretas.
- **Revocação:** dos objetos reais, quantos foram encontrados.
- **mAP@0.5:** média das precisões médias considerando IoU de 0,5.
- **mAP@0.5:0.95:** média do mAP para dez limiares de IoU (0,50 a 0,95), portanto mais rigorosa.

Essas métricas se relacionam diretamente com a matriz de confusão e com a curva precisão-revocação, e são a base da análise de resultados na Seção 4.

3 METODOLOGIA

Esta seção descreve a coleta, a definição e anotação do alvo, o pré-processamento, a divisão dos dados e a estratégia de treinamento.

3.1 COLETA DE DADOS

A coleta foi feita de forma programática a partir do serviço público de tiles XYZ do ESRI World Imagery, que oferece cobertura submétrica de São Paulo. Para o heliponto usei o zoom 19 (cerca de 0,27 metros por pixel), o máximo prático para a

região nesse serviço. Cada tile nativo tem 256 por 256 pixels; para ganhar contexto e qualidade de visualização, juntei blocos de 2 por 2 tiles adjacentes, formando blocos de 512x512 pixels, com um arquivo de georreferenciamento (world file) gerado para cada bloco.

As regiões varridas foram escolhidas pela densidade de helipontos (corredor Faria Lima, Itaim, Vila Olímpia, Berrini, Brooklin, Morumbi e adjacências) e documentadas por bairro, com a bounding box e o zoom registrados. Após a coleta, cada bloco passou por curadoria manual: separei os blocos com heliponto dos blocos sem heliponto, descartando os irrelevantes.

```
# bbox no formato (lon_min, lat_min, lon_max, lat_max), igual ao enunciado
# papel define o destino no split por bairro: treino_val ou teste
REGIOES = {
    'itaim_faria_lima': {'bbox': (-46.694, -23.592, -46.672, -23.572), 'papel': 'treino_val'},
    'vila_olimpia': {'bbox': (-46.695, -23.604, -46.676, -23.590), 'papel': 'treino_val'},
    'brooklin_berrini': {'bbox': (-46.703, -23.624, -46.683, -23.606), 'papel': 'treino_val'},
    'avenida_paulista': {'bbox': (-46.660, -23.572, -46.640, -23.555), 'papel': 'teste'},
}

# definicao isolada dos bairros novos, no formato (lon_min, lat_min, lon_max, lat_max)
REGIOES = {
    'pinheiros': {'bbox': (-46.710, -23.575, -46.696, -23.560), 'papel': 'treino_val'},
    'morumbi_jk': {'bbox': (-46.730, -23.612, -46.708, -23.590), 'papel': 'treino_val'},
    'chucui_zaidan': {'bbox': (-46.726, -23.628, -46.706, -23.614), 'papel': 'treino_val'},
    'santo_amaro': {'bbox': (-46.716, -23.648, -46.700, -23.630), 'papel': 'treino_val'},
}

REGIOES = {
    'moema': {'bbox': (-46.670, -23.605, -46.655, -23.590), 'papel': 'treino_val'},
    'campo_belo': {'bbox': (-46.676, -23.625, -46.661, -23.610), 'papel': 'treino_val'},
    'centro': {'bbox': (-46.645, -23.550, -46.630, -23.530), 'papel': 'treino_val'},
    'higienopolis': {'bbox': (-46.665, -23.548, -46.650, -23.535), 'papel': 'treino_val'},
    'pacaembu_perdizes': {'bbox': (-46.685, -23.545, -46.665, -23.530), 'papel': 'treino_val'},
}

REGIOES = {
    'faria_lima_norte': {'bbox': (-46.695, -23.572, -46.678, -23.560), 'papel': 'treino_val'}
}
```

Figura 2 - Bounding Boxes das Regiões Utilizadas

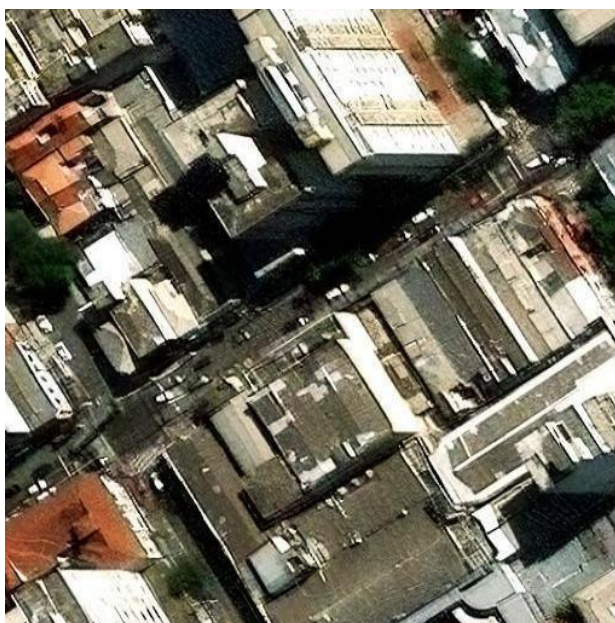


Figura 3 - Tile 512x512

3.2 DEFINIÇÃO DO ALVO E ANOTAÇÃO

O objeto-alvo escolhido foi o heliponto (classe única chamada heliponto). São Paulo possui uma das maiores frotas de helicópteros urbanos do mundo, e a marcação padrão de heliponto é um alvo visualmente distinto e com forma geométrica clara em vista de topo, o que favorece a anotação e o aprendizado. Por ser pequeno, exigiu a coleta em alta resolução discutida na seção anterior.

A anotação foi feita no Roboflow, com uma única classe, seguindo um padrão escrito de anotação que cobre a margem máxima entre a caixa e o objeto, o tratamento de helipontos cortados pela borda (anota-se apenas a porção visível) e o tratamento de sombra e reflexo. As anotações foram exportadas no formato YOLO (um arquivo de texto por imagem, com coordenadas normalizadas), e as imagens sem heliponto foram mantidas como exemplos negativos (arquivo de rótulo vazio) para reduzir falsos positivos.

Ao todo foram reunidas aproximadamente 230 instâncias anotadas de heliponto.

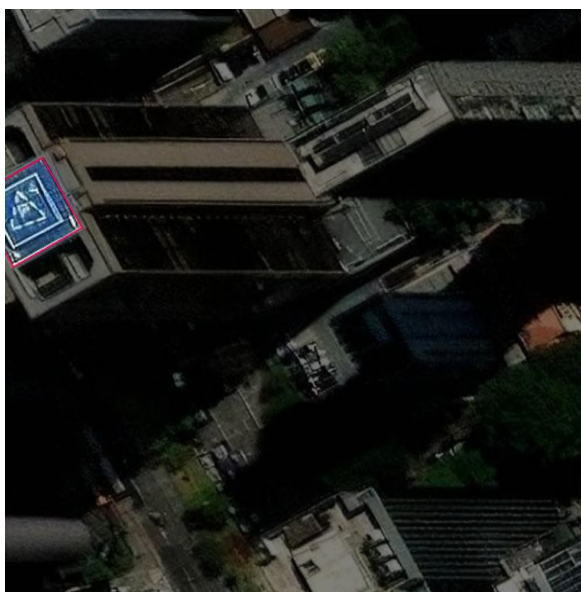


Figura 4 - Imagem com Anotação de Heliponto do Roboflow

3.3 PRÉ-PROCESSAMENTO E DIVISÃO DOS DADOS

No Roboflow, cada bloco de 512 por 512 foi redimensionado para 640 por 640 pixels, resolução de entrada do modelo. A divisão dos dados foi feita por holdout geográfico (por bairro), e não por sorteio aleatório: as regiões de treino e validação usaram um conjunto de bairros, enquanto a Avenida Paulista foi reservada inteira

como bairro inédito de teste. Essa escolha evita o vazamento de tiles visualmente parecidos entre treino e teste e fornece uma medida honesta de generalização.

A augmentation foi aplicada somente ao conjunto de treino (rotação de 90 graus e variação de brilho de mais ou menos 20 por cento); validação e teste ficaram sem augmentation, conforme o conceito visto em aula. Os volumes finais foram 538 imagens de treino (Pós Augmentation), 60 de validação e 195 de teste (Alto volume devido a grande quantidade de imagens sem Helipontos).

3.4 ESTRATÉGIA DE TREINAMENTO

O treino partiu de pesos pré-treinados (transfer learning), com o modelo principal YOLO26n. Para garantir reprodutibilidade, fixei uma semente (seed = 42) e registrei todos os hiperparâmetros. Foram executadas duas rodadas de treinamento variando exatamente um hiperparâmetro entre elas (o número de épocas: 30 na primeira e 50 na segunda), mantendo o restante idêntico, para uma comparação justa. A melhor rodada foi selecionada pela mAP@0.5:0.95 de validação.

Como comparação de arquitetura, treinei adicionalmente o YOLO11n sob os mesmos hiperparâmetros da melhor rodada, avaliando ambos no mesmo conjunto de teste, única forma de comparação válida, já que a mAP não é comparável entre datasets ou divisões diferentes.

Rodada	Modelo	Qtd Épocas	Batch	imgsz	seed	MAP@0.5	MAP@0.5.95
1	YOLO26n	30	16	640	42	0.946	0.729
2	YOLO26n	50	16	640	42	0.954	0.778

4 RESULTADOS E DISCUSSÃO

As métricas foram calculadas no conjunto de teste (Avenida Paulista) com limiar de confiança de 0,25 e IoU de 0,50.

4.1 COMPARAÇÃO DAS RODADAS E DAS ARQUITETURAS

Resultados do modelo principal (YOLO26n) e da arquitetura de comparação (YOLO11n) no conjunto de teste (bairro inédito), sob os mesmos hiperparâmetros:

Modelo	mAP@0.5	mAP@0.5:0.95	Precisão	Revocação
YOLO26n	0.709	0.530	0.710	0.817
YOLO11n	0.781	0.536	0.869	0.818

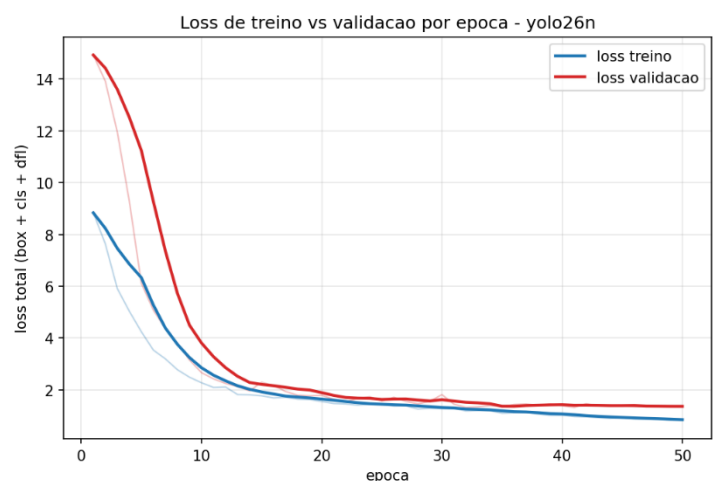
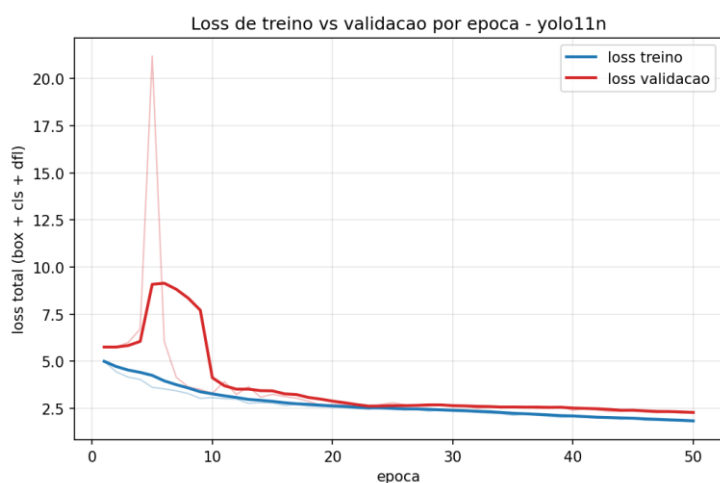
Na avaliação final, o YOLO11n apresentou desempenho superior ao YOLO26n no conjunto de teste: precisão de 0,869 contra 0,710 e $mAP@0.5$ de 0,781 contra 0,709, com revocação praticamente igual (0,818 contra 0,817) e $mAP@0.5:0.95$ equivalente (0,536 contra 0,530). Cabe registrar que, em versões anteriores do conjunto de dados, a ordem entre os dois modelos se invertia, com o YOLO26n à frente. Essa alternância é coerente com a alta variância de um conjunto de teste com apenas 33 instâncias, em que diferenças de poucos pontos de mAP não são estatisticamente significativas.

Mantive o YOLO26n como modelo principal por ter sido a escolha de projeto a priori e por suas melhorias voltadas a objetos pequenos, mas, para este problema e com este volume de dados, os dois modelos nano devem ser considerados equivalentes.

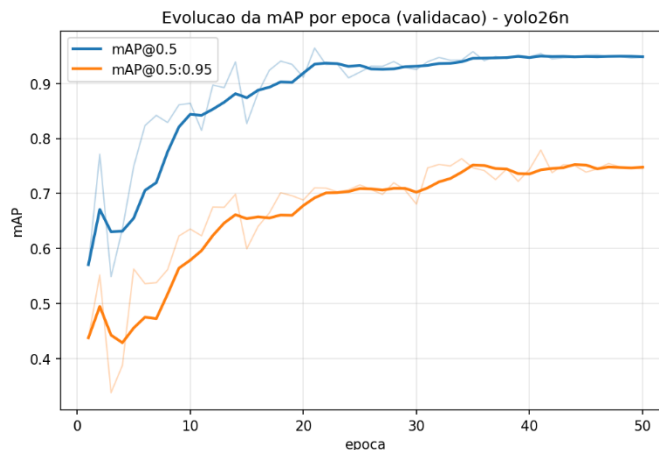
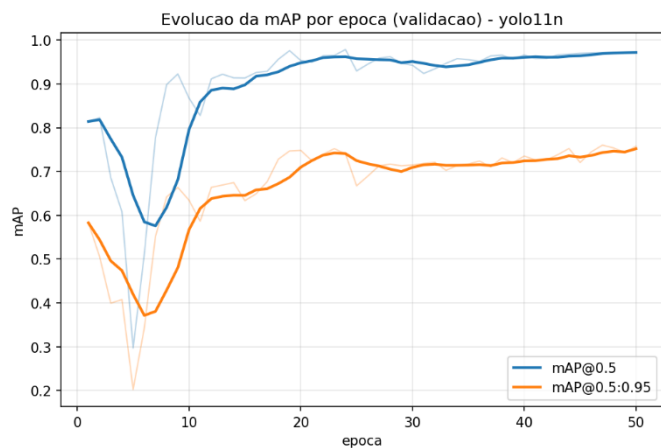
O conjunto de teste foi varrido por inteiro, então a maioria dos tiles é de fundo, sem o alvo. Isso, somado ao número limitado de instâncias, aumenta a variância das métricas, que devem ser lidas com cautela.

4.2 CURVAS DE TREINO

As curvas abaixo, geradas a partir do histórico de treino da melhor rodada, mostram a evolução por época. A mAP (validação) cresce e estabiliza conforme o modelo aprende, enquanto as perdas de treino e validação decrescem; a distância entre as duas curvas de perda indica o nível de overfitting.



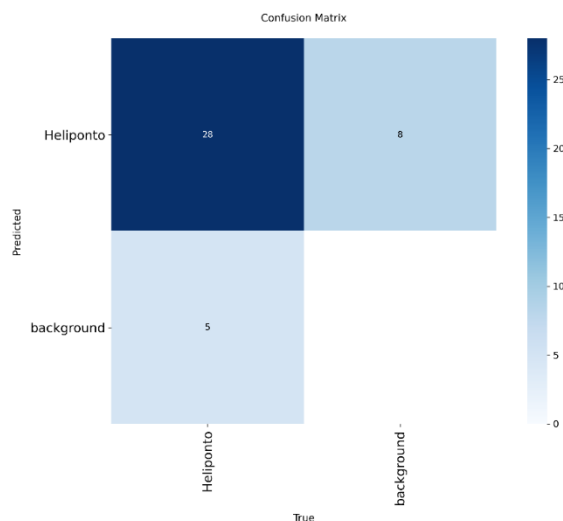
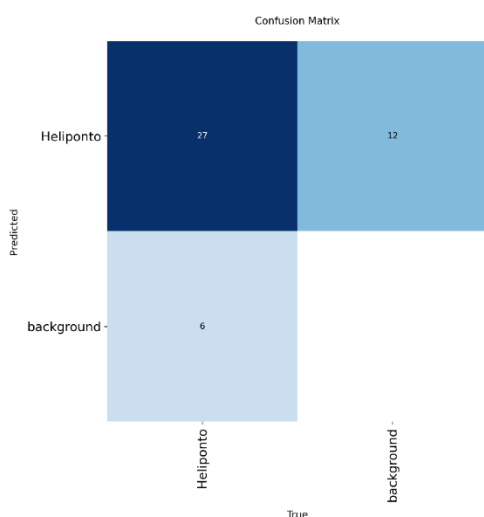
Figuras 5 e 6 - Gráficos de Loss por Época



Figuras 7 e 8 - Gráficos da métrica mAP por época

4.3 ANÁLISE DE ERROS

Casando as detecções (confiança $\geq 0,25$) com as anotações verdadeiras por $\text{IoU} \geq 0,50$, coletei exemplos representativos do comportamento do modelo no teste. A matriz de confusão, no ponto de confiança 0,25, registrou:



Figuras 9 e 10 - Matrizes de Confusão, 1º YOLO26 e 2º YOLO11

Discussão das causas prováveis:

- **Falsos positivos:** marcas circulares em coberturas (caixas d'água, marcações de estacionamento, formatos quadrados e etc) que se parecem com o padrão do heliponto.
- **Falsos negativos:** helipontos com sombra, baixo contraste, pintura desgastada ou parcialmente cobertos por equipamentos, além de casos cortados pela borda do bloco.

Vale notar que a matriz de confusão é tomada em um limiar fixo de confiança (0,25), mais baixo, enquanto a precisão e a revocação reportadas na Seção 4.1

correspondem ao ponto de operação que maximiza o F1. Essa diferença de limiar explica por que a matriz pode aparentar mais falsos positivos do que a precisão sugere: boa parte desses falsos positivos tem confiança baixa e desaparece ao subir o limiar.

4.4 INFERÊNCIA EM BAIRRO INÉDITO

Para verificar a generalização, rodei a inferência sobre os tiles da Avenida Paulista, bairro não usado no treino, e salvei as imagens com as detecções. A Avenida Paulista tem características visuais diferentes das regiões de treino (edifícios mais antigos, hotéis), o que estressa o modelo e mostra até que ponto ele generaliza para um domínio novo.

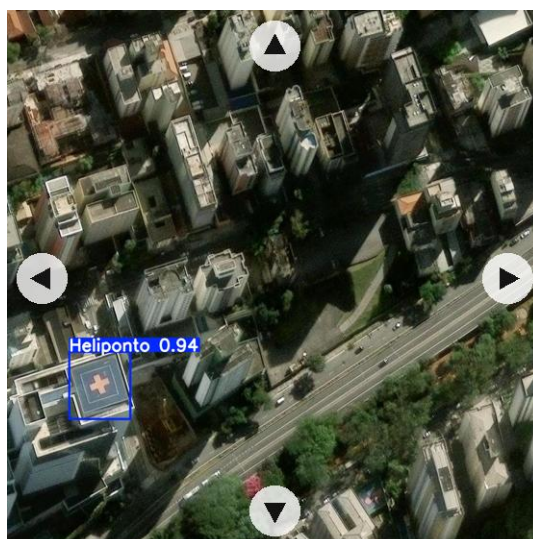


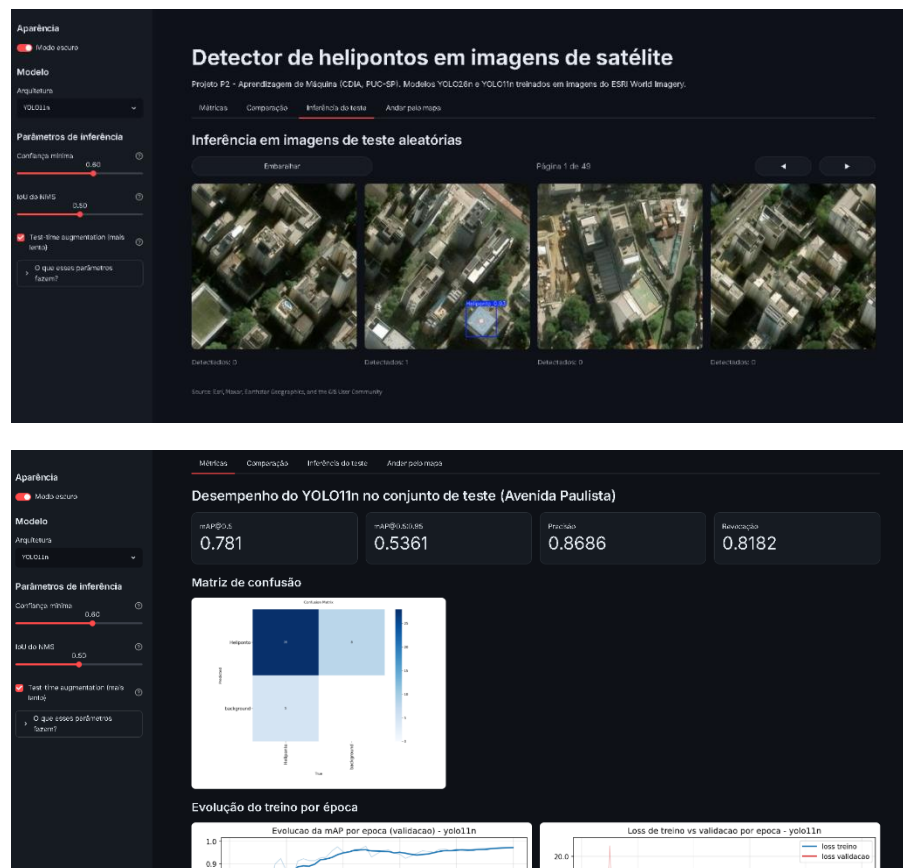
Figura 11 - Imagem da Inferência no recorte da Paulista

5 APLICAÇÃO NO STREAMLIT

Para melhor visibilidade e teste das funcionalidades, desenvolvi uma aplicação com interface visual em Streamlit, disponível em mapsyolo26puc.streamlit.app e alimentada diretamente pelo repositório GitHub do projeto. A aplicação carrega o melhor modelo e oferece três áreas: uma página de métricas (com a matriz de confusão e as curvas de treino), a inferência em imagens de teste aleatórias exibidas em um carrossel de quatro por vez, e um modo de navegar por um mosaico de bairro com setas desenhadas dentro da própria imagem (zoom fixo, no estilo de mapas interativos), com o modelo detectando helipontos em tempo real na janela visível.

A aplicação permite ainda variar o limiar de confiança e o IoU do NMS, na linha da aula de métricas, e oferece a opção de test-time augmentation na inferência. Optei

por não incluir upload de imagem para evitar entradas fora do domínio do modelo durante a apresentação.



Figuras 12 e 13 - Telas da aplicação Streamlit

6 CONCLUSÕES

O projeto cobriu o ciclo completo de um detector de objetos em imagens de satélite, com ênfase na construção e curadoria do dataset. No conjunto de teste (Avenida Paulista, bairro inédito), o modelo principal (YOLO26n) encontrou cerca de 82% dos helipontos (revocação 0,817), com precisão de 0,710 e mAP@0.5 de 0,709. A arquitetura de comparação (YOLO11n) atingiu mAP@0.5 de 0,781 e precisão de 0,869 no mesmo teste, ficando à frente nesta versão do dataset, diferença dentro da margem de variância de um teste pequeno. A divisão por bairro forneceu uma medida honesta de generalização.

A principal lição confirmada foi a de que o resultado é limitado pelos dados, e não pela arquitetura: trocar de modelo (nano entre YOLO26 e YOLO11) ou ampliarmarginalmente a base não moveu o resultado de forma significativa. Ganhos reais dependeriam de um aumento substancial do volume de dados anotados.

As ferramentas de IA generativa foram utilizadas como apoio na construção da aplicação Streamlit e na revisão pontual de código e texto. A definição do alvo, a coleta, a anotação, as decisões técnicas e a análise dos resultados foram de minha autoria, com base nos conteúdos abordados em sala de aula.

REFERÊNCIAS

ULTRALYTICS. Documentação do YOLO (YOLO26, YOLO11, YOLOv8). Disponível em: <https://docs.ultralytics.com/>. Acesso em: jun. 2026.

ROBOFLOW. Documentation. Disponível em: <https://docs.roboflow.com/>. Acesso em: jun. 2026.

ESRI. World Imagery. Source: Esri, Maxar, Earthstar Geographics, and the GIS User Community.

MATERIAIS DA DISCIPLINA. Notebooks de localização e detecção, YOLO e métricas, formatos e ferramentas, e notas de CNN em PyTorch. Aprendizagem de Máquina (CDIA), PUC-SP, 2026.